

레고 블록 실시간 탐색 앱

서비스 파이프라인 기획서 (외주 개발 의뢰용)

버전 1.0 | 작성일 2026-05-08

1. 프로젝트 개요

1.1 한 줄 요약

스마트폰 카메라로 어지러운 레고 블록 더미를 비추면, 사용자가 원하는 특정 부품의 위치를 실시간 AR로 하이라이트해 주는 **비전 기반 실시간 레고 블록 매칭 모바일 앱**.

1.2 해결하는 문제

기존 상황	본 서비스 이후
조립 설명서를 보며 필요한 부품을 방바닥에서 하나하나 눈으로 찾음	설명서를 카메라로 찍거나 음성으로 말하면 해당 부품 위치를 AR로 표시
수백 개 부품이 뒤섞인 상황에서 탐색 시간이 수 분 소요	실시간(30fps 목표)으로 목표 블록 추적
기존 앱(Brickit, Pileometer)은 '재고 관리·창작 추천'에 초점, 즉각적인 탐색 불가	사전 준비 없이, 바닥 위 혼잡한 환경에서도 즉시 사용 가능

1.3 경쟁 서비스와의 차별점

서비스	핵심 기능	한계
Brickit	전체 브릭 스캔 후 창작품 추천	조립 중 개별 부품 탐색 불가
Pileometer	흰 바탕 위 부품 라벨링·재고 관리	흰 배경 환경 필수, 실시간 AR 없음
본 서비스	복잡한 현실 배경에서 단일 목표 부품 실시간 AR 추적	—

2. 타겟 사용자 및 사용 시나리오

2.1 타겟 사용자

- 레고 조립을 즐기는 **개인 취미 사용자** (초등학생 ~ 성인)
- 레고 클럽·교육 환경에서 부품을 공동 사용하는 **교육 기관**

- 대량의 벌크 레고를 보유한 수집가·리셀러

2.2 핵심 사용 시나리오

[시나리오 A - 설명서 기반 탐색]

1. 사용자가 조립 설명서를 펼침
2. 앱을 열어 설명서의 특정 부품 도면을 카메라로 촬영 (쿼리 이미지 입력)
3. 앱이 해당 부품을 DB에서 매칭
4. 바닥에 흩어진 레고를 카메라로 비추면 목표 블록 테두리가 빛나는 AR로 표시
5. 사용자가 해당 부품을 집어 조립

[시나리오 B - 음성/자연어 기반 탐색]

1. 사용자가 "빨간색 2x3 기본 브릭 찾아줘" 라고 말하거나 텍스트 입력
2. 이후 과정은 시나리오 A와 동일

[시나리오 C - 리스트 기반 탐색]

1. 사용자가 Rebrickable API 연동으로 특정 레고 세트의 부품 목록을 불러옴
2. 목록에서 원하는 부품 선택
3. AR 탐색 실행

3. 시스템 아키텍처 전체 개요

[사용자 입력]

- └─ 자연어/음성 텍스트
- └─ 쿼리 이미지 (설명서 사진)
- └─ 부품 리스트 선택 (Rebrickable 연동)



[멀티모달 쿼리 인코더] — CLIP 계열 Vision-Language Model

→ 프롬프트 임베딩 벡터 생성



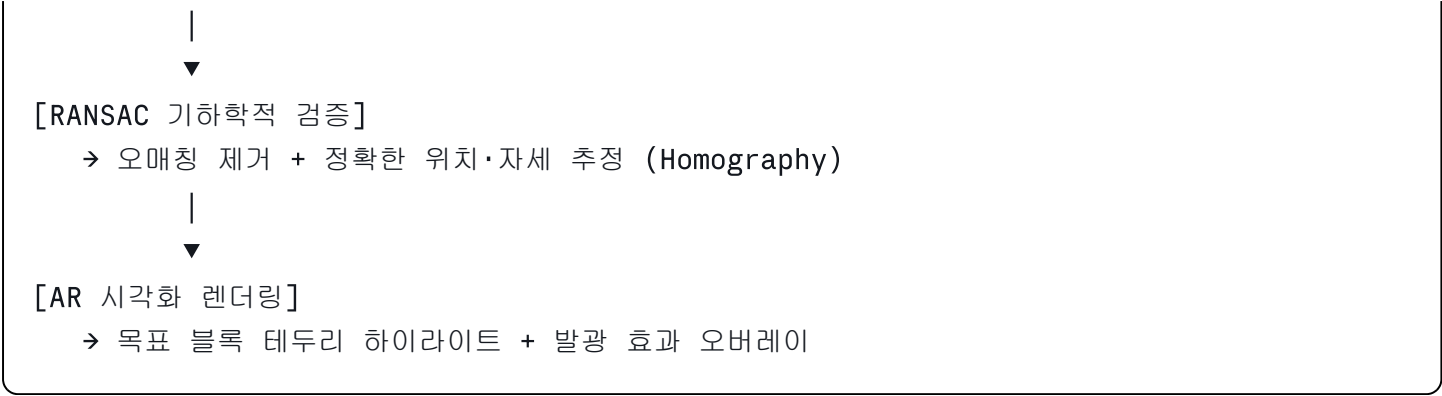
[Rebrickable DB + 오프라인 HNSW 인덱스]

→ 타겟 부품 공식 3D 이미지 + 특징점 벡터 DB 확보



[실시간 카메라 스트림 처리 루프 - 목표: 30fps]

- └─ MobileNet 백본 → 시각 특징 맵 추출
- └─ Cross-Modal Attention (프롬프트 임베딩 × 특징 맵)
 - ROI 확률 히트맵 생성
- └─ 히트맵 이진화 + Dilation(팽창) 후처리 → ROI 마스크
- └─ ROI 마스크 내 ORB 특징점 추출
- └─ HNSW 인덱스 질의 → 후보 매칭



4. 기능 명세

4.1 쿼리 입력 모듈

기능	상세 설명	우선순위
설명서 이미지 촬영 쿼리	카메라로 부품 도면 촬영 → 자동 크롭·전처리 후 쿼리 이미지로 사용	P0
자연어 텍스트 쿼리	"빨간 2x4 브릭" 등 텍스트 입력 → CLIP 텍스트 인코더 처리	P0
음성 인식 쿼리	디바이스 STT(Speech-to-Text) API 활용, 텍스트 쿼리로 변환	P1
Rebrickable 세트 연동	API로 세트 부품 목록 불러온 뒤 리스트에서 선택	P1
최근 쿼리 히스토리	직전 탐색 부품 5개 저장, 재선택 가능	P2

4.2 데이터베이스 및 인덱싱 모듈

기능	상세 설명	우선순위
Rebrickable API 연동	부품 메타데이터(ID, 색상, 카테고리), 공식 3D 렌더링 이미지 수집	P0
오프라인 HNSW 인덱스 구축	서버 사이드에서 사전 구축 후 앱에 번들 또는 CDN 다운로드	P0
자주 사용 부품 캐싱	상위 사용 빈도 기본 브릭(2x2, 2x4 등) 특징점 로컬 캐시	P1
DB 증분 업데이트	신규 레고 부품 출시 시 HNSW 인덱스에 데이터 추가(재학습 불필요)	P1

4.3 비전 AI 추론 모듈

기능	상세 설명	우선순위
MobileNet 백본 특징 추출	모바일 최적화 경량 백본으로 실시간 특징 맵 생성	P0
Cross-Modal Attention ROI 예측	프롬프트 임베딩과 특징 맵 내적 → ROI 히트맵 생성	P0
ROI 마스크 Dilation 후처리	카메라 흔들림 대응, 마스크 팽창 처리	P0
ORB 특징점 추출 (마스크 내)	ROI 영역 한정 특징점 추출로 연산량 절감	P0
HNSW 고속 ANN 검색	사전 구축 인덱스에 쿼리 → 최근접 이웃 탐색	P0
RANSAC 기하학적 검증	Homography 추정, 오매칭 제거, 위치·자세 확정	P0
HSL 전처리 (조명 보정)	특징점 추출 전 HSL 변환으로 조명·반사 노이즈 최소화	P1
색상 주조색 분리 필터	보조색 혼선 방지, 주조색 기반 오인식 억제	P1
부분 가림 보완 (보조 지표 융합)	색상 분포 등 추가 신호로 가림 상황 매칭률 보완	P2

4.4 AR 시각화 모듈

기능	상세 설명	우선순위
실시간 테두리 하이라이트	RANSAC으로 확정된 블록 외곽에 발광 테두리 오버레이	P0
발광(Glow) 이펙트	부품 색상에 맞는 컬러 발광 효과 렌더링	P0
탐색 성공/실패 피드백	매칭 성공 시 햅틱+사운드 피드백, 미발견 시 안내 메시지	P1
복수 후보 표시	동일 부품이 여러 개인 경우 모두 하이라이트 + 개수 표시	P1
프리즈 프레임 핀 기능	탐색 결과를 정지 화면에 핀으로 고정해 이동 편의 제공	P2

5. 학습 데이터 및 모델 학습 명세

5.1 학습 데이터셋 구성

- 구조: [[입력 이미지, 타겟 프롬프트, 정답 마스크(GT)]] 트리플렛(Triplet) 형태
- 배경 이미지: 실제 방바닥, 책상 등 복잡한 배경 사진 (다양한 조명 환경 포함)
- 부품 이미지: Rebrickable API의 공식 3D 렌더링 이미지
- 데이터 증강:

- Copy-Paste 합성: 배경 위에 3D 렌더링 이미지를 다양한 각도·크기로 합성
- 조명 변환: 밝기, 채도, 색온도 랜덤 변환
- 부분 가림(Occlusion) 합성: 무작위 블록 이미지로 일부 가리기

5.2 학습 목표 및 손실 함수

- **목표:** 모델이 프롬프트 의미를 해석해 목표 블록 영역의 확률 점수를 1에 가깝게 예측
- **손실 함수:** Binary Cross Entropy Loss + Dice Loss 융합
 - BCE: 픽셀 단위 정답 여부
 - Dice Loss: 예측 마스크와 GT 마스크 간 겹침(IoU) 최적화

5.3 모델 경량화 요구사항

- 모바일 실시간 처리(목표 30fps) 위한 **TensorFlow Lite 또는 Core ML 변환** 필수
- 백본: **MobileNetV3** 또는 동급 경량 모델 사용
- 추론 지연: 단일 프레임 처리 **33ms 이하** 목표 (GPU 가속 사용 시)

6. 기술 스택 권고안

6.1 모바일 앱

항목	권고 기술	비고
플랫폼	iOS / Android 동시 지원	React Native 또는 Flutter
카메라 스트림	Native Camera API	AVFoundation(iOS), Camera2(Android)
AR 렌더링	ARKit(iOS) / ARCore(Android) 또는 OpenCV	경량화 시 OpenCV 우선
온디바이스 추론	TensorFlow Lite / Core ML	HNSW: hnswlib 바인딩
음성 인식	디바이스 내장 STT API	iOS SFSpeechRecognizer, Android SpeechRecognizer

6.2 서버/백엔드

항목	권고 기술	비고
API 서버	Python FastAPI	모델 서빙, Rebrickable 프록시
DB	PostgreSQL + pgvector 또는 Qdrant	부품 메타데이터 + 벡터 검색
HNSW 인덱스	hnswlib (Python)	오프라인 사전 구축 후 앱 번들/CDN 제공
CLIP 인코딩 서버	PyTorch + HuggingFace Transformers	쿼리 임베딩 생성
스토리지	AWS S3 또는 GCS	부품 이미지, 인덱스 파일
배포	Docker + AWS ECS 또는 GCP Cloud Run	

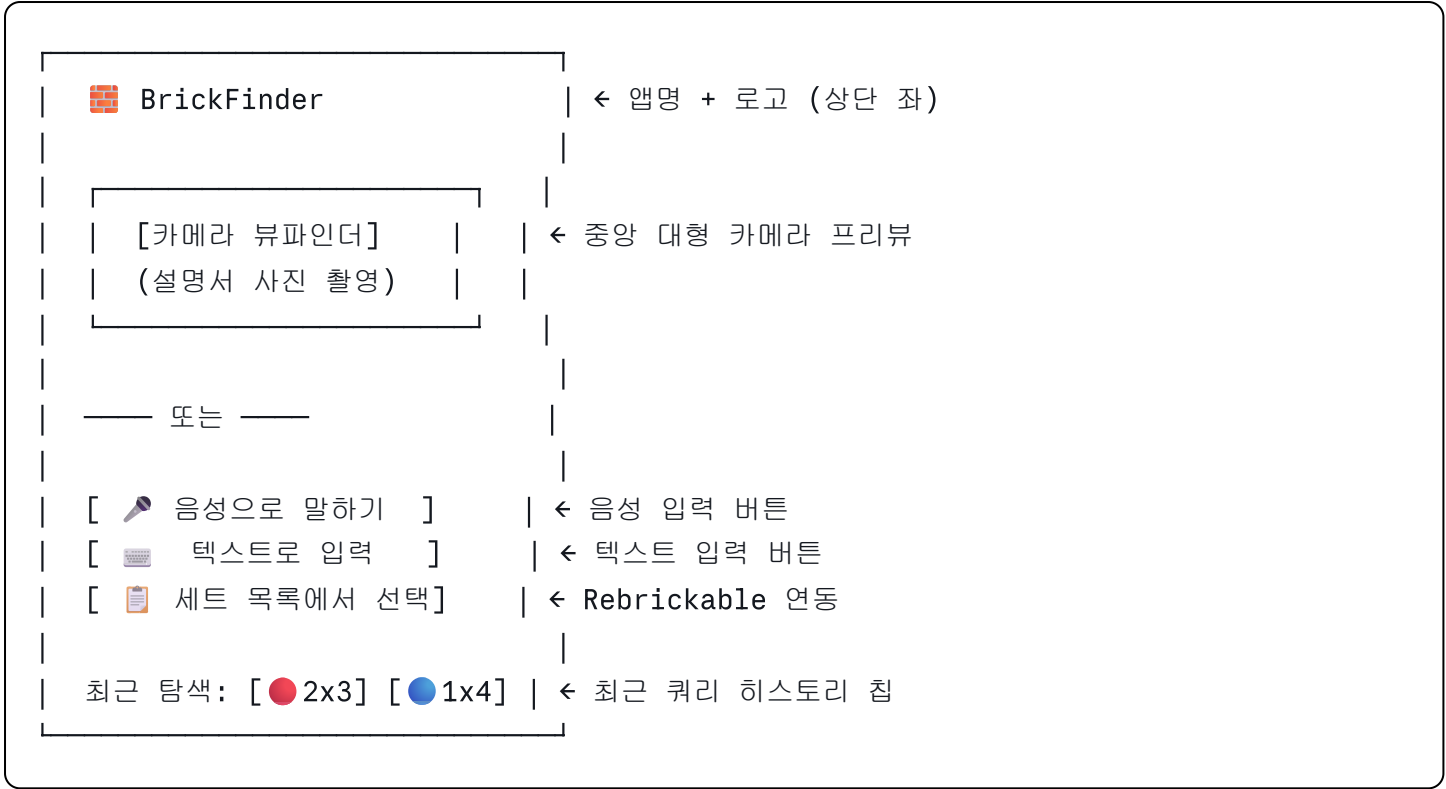
7. 화면 설계 (UI/UX)

7.1 핵심 디자인 원칙

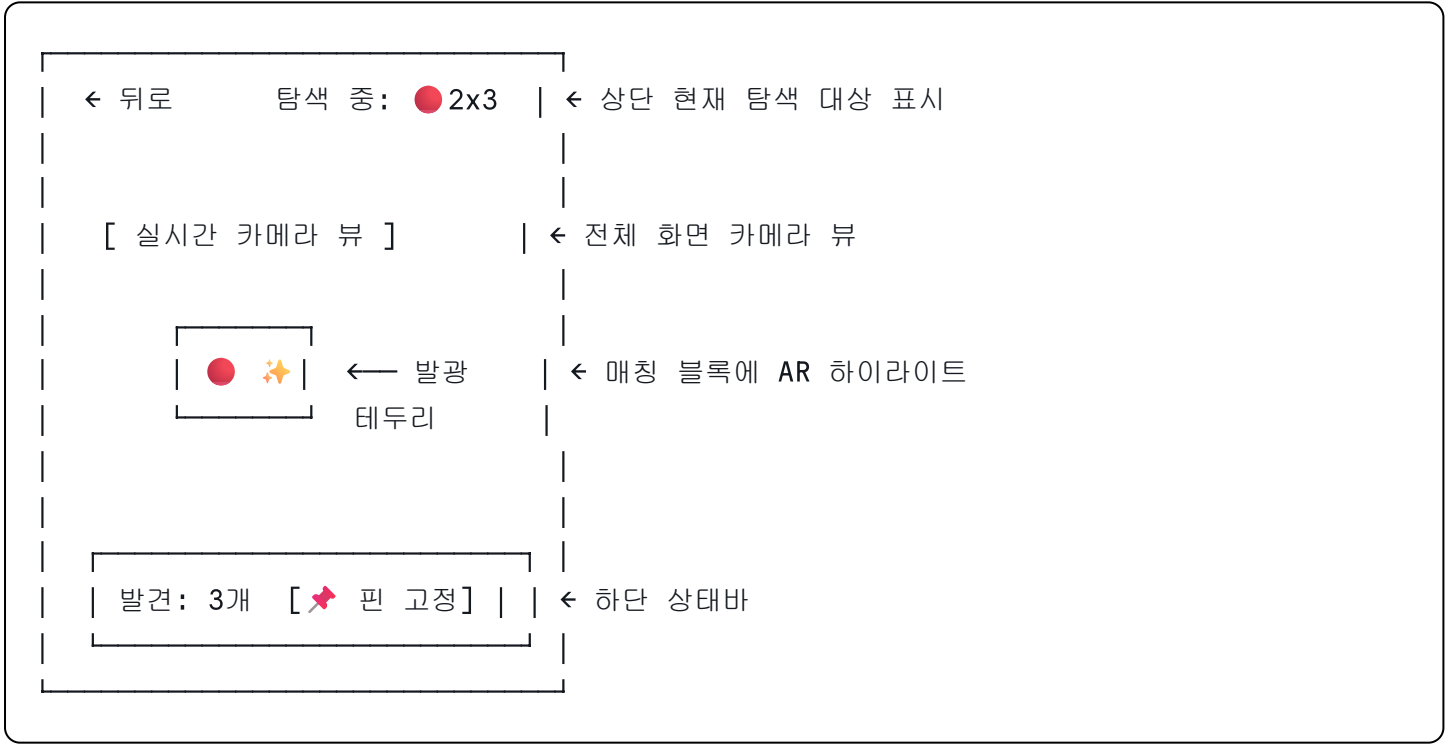
- 단순함 최우선: 조립 중에 앱을 조작하므로 최소 탭 수로 핵심 기능 도달
- 다크 모드 기본: 화면이 카메라 뷰어가 주를 이루므로 다크 테마 적용
- 레고 브랜드 감성: 원색 계열 포인트 컬러, 둥근 모서리, 장난감처럼 친근한 인터랙션
- 한 손 조작 최적화: 하단 중심 UI 배치

7.2 화면 목록 및 설명

[화면 1] 홈 / 쿼리 입력 화면



[화면 2] 탐색 (AR 뷰어) 화면



[화면 3] 부품 상세 / 매칭 결과 화면

← 탐색으로 돌아가기

[부품 공식 3D 렌더링 이미지]

이름: 기본 브릭 2x3

색상: 빨간색

파트 번호: 3002

카테고리: 기본 브릭

[ 이 부품으로 탐색 시작] ← CTA 버튼

[화면 4] 세트 연동 화면 (Rebrickable)

세트 번호 입력 또는 검색

[75257-1 ]

 스타워즈 밀레니엄팔콘

총 부품: 1,329개

부품 목록

 기본 브릭 2x4 ×12

 플레이트 1x2 ×24

 타일 2x2 ×8

[선택한 부품으로 탐색]

7.3 컬러 팔레트

용도	색상	비고
배경	 #1A1A2E	딥 네이비 다크
카드/패널	 #16213E	
포인트 (CTA)	 #E94560	레고 레드 계열
AR 하이라이트 기본	부품 색상 연동	빨간 블록 → 빨간 발광
텍스트 메인	 #EAEAEA	
텍스트 서브	 #8892B0	

8. 성능 목표 (KPI)

지표	목표값	측정 방법
AR 탐색 프레임 레이트	≥ 30fps	기기 내 FPS 측정
단일 프레임 추론 지연	≤ 33ms	처리 시간 로깅
ROI 예측 정확도 (IoU)	≥ 0.75	테스트셋 평가
HNSW 검색 소요 시간	≤ 10ms	벤치마크 측정
매칭 Top-1 정확도 (단색 배경)	≥ 90%	정제 환경 테스트
매칭 Top-1 정확도 (복잡 배경)	≥ 75%	실제 환경 테스트
앱 콜드 스타트 시간	≤ 3초	실기기 측정

9. 개발 단계별 로드맵

Phase 1 — 핵심 파이프라인 MVP (우선 납품 목표)

- ☐ Rebrickable API 연동 및 부품 데이터 수집 스크립트
- ☐ HNSW 인덱스 오프라인 구축 파이프라인
- ☐ CLIP 기반 텍스트 쿼리 → 임베딩 변환 서버
- ☐ MobileNet + Cross-Modal Attention ROI 예측 모델 학습
- ☐ ORB 특징점 추출 + HNSW 매칭 + RANSAC 검증 파이프라인
- ☐ 카메라 스트림 + AR 테두리 하이라이트 (기본 발광 효과)
- ☐ 텍스트 쿼리 입력 UI

Phase 2 — 품질 고도화

- ☐ 설명서 이미지 쿼리 (카메라 촬영 → 자동 매칭)
- ☐ 음성 입력 연동
- ☐ HSL 조명 보정 전처리 적용
- ☐ 색상 주조색 분리 필터 적용
- ☐ 복수 후보 동시 하이라이트

Phase 3 — 사용성 완성

- ☐ Rebrickable 세트 연동 UI
- ☐ 부분 가림 보완 알고리즘
- ☐ 자주 사용 부품 로컬 캐싱
- ☐ 탐색 히스토리, 핀 고정 기능
- ☐ 앱스토어 배포 준비 (심사 대응)

10. 납품 산출물 목록

산출물	형식
모바일 앱 소스코드	GitHub Repository
AI 모델 파일	<code>.tflite</code> / <code>.mlmodel</code>
HNSW 인덱스 구축 스크립트	Python
백엔드 API 서버 소스코드	Python (FastAPI)
Docker Compose 배포 파일	<code>.yaml</code>
데이터 수집·증강 스크립트	Python
API 문서	Swagger / Markdown
테스트 결과 보고서	PDF

11. 참고 자료 및 외부 의존성

- Rebrickable API:** <https://rebrickable.com/api/> (부품 데이터, 3D 이미지)
- CLIP 모델:** OpenAI CLIP 또는 HuggingFace `openai/clip-vit-base-patch32`
- HNSW 라이브러리:** `hnswlib` (Python/C++)
- 특징점 추출:** OpenCV ORB
- 기하학적 검증:** OpenCV RANSAC + findHomography
- 경량 백본:** `MobileNetV3` (TensorFlow / PyTorch)

문의 및 협의 사항: 본 기획서의 세부 요구사항 중 기술적으로 조정이 필요한 항목은 착수 전 협의 후 반영합니다. Phase 1 MVP 기준 일정 및 견적을 우선 회신해 주시면 감사하겠습니다.

